list Datatype

Lists in Python are similar to arrays in C or Java. A list represents a group of elements. The main difference between a list and an array is that a list can store different types of elements but an array can store only one type of elements. Also, lists can grow dynamically in memory. But the size of arrays is fixed and they cannot grow at runtime. Lists are represented using square brackets [ ] and the elements are written in [ ], separated by commas. For example,

```
list = [10, -20, 15.5, 'Vijay', "Mary"]
```

will create a list with different types of elements. The slicing operation like [0: 3] represents elements from 0th to 2nd positions, i.e. 10, 20, 15.5. Have a look at some of the operations on a list in the following IDLE window, as shown in Figure 3.8:

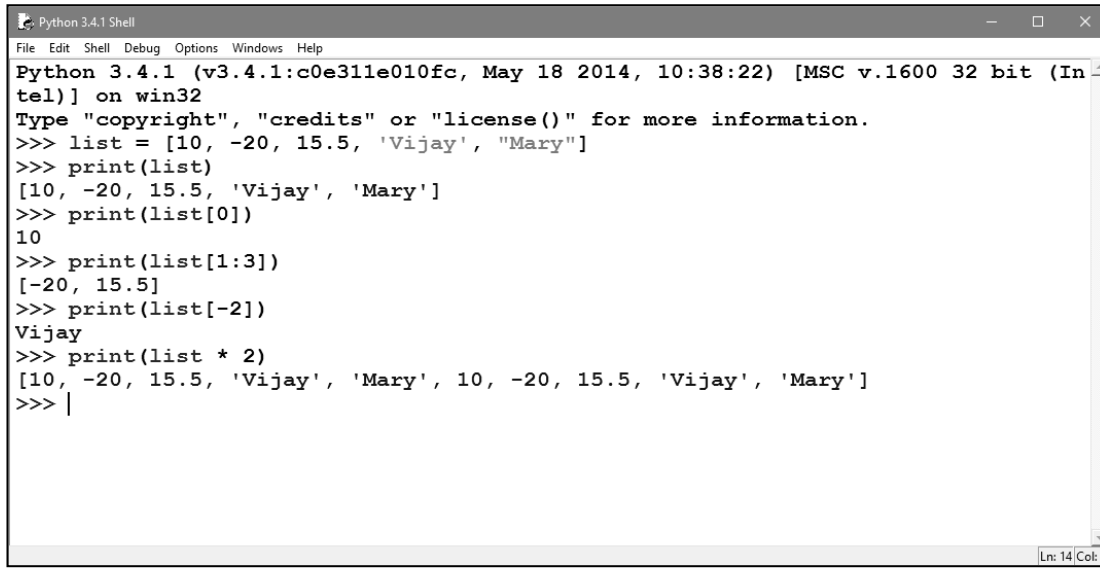


Figure 3.8: Some Operations on Lists

tuple Datatype

A tuple is similar to a list. A tuple contains a group of elements which can be of different types. The elements in the tuple are separated by commas and enclosed in parentheses ( ). Whereas the list elements can be modified, it is not possible to modify the tuple elements. That means a tuple can be treated as a read-only list. Let's create a tuple as:

```
tpl = (10, -20, 15.5, 'Vijay', "Mary")
```

The individual elements of the tuple can be referenced using square braces as tpl[0], tpl[1], tpl[2], … Now, if we try to modify the 0th element as:

```
tpl[0] = 99
```

This will result in error. The slicing operations which can be done on lists are also valid in tuples. See the general operations on the tuple in Figure 3.9:

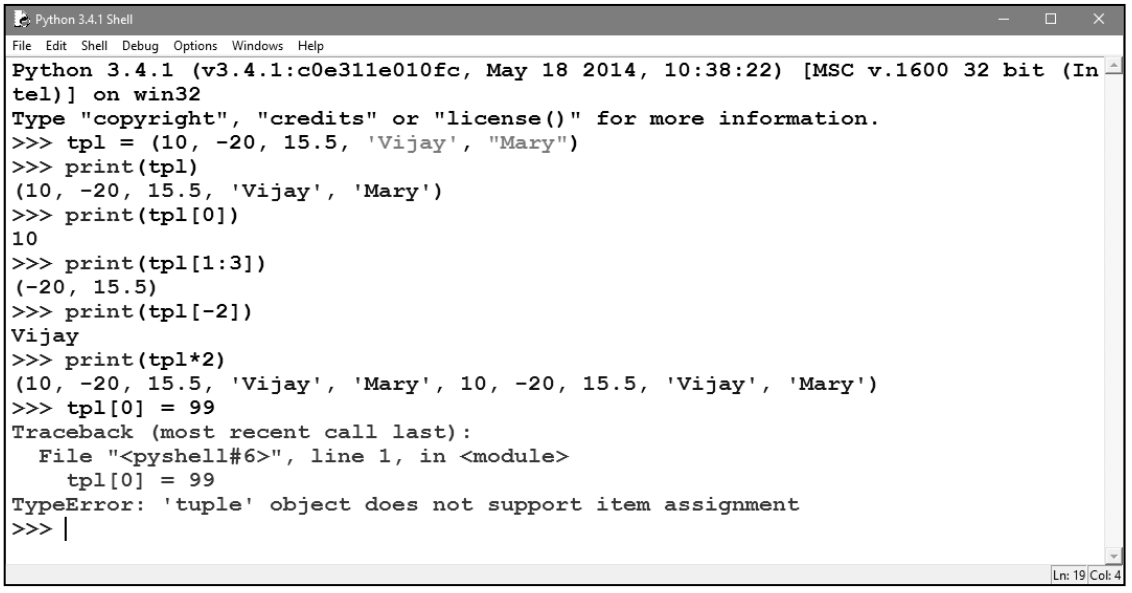


Figure 3.9: Some operations on tuples

range Datatype

The range datatype represents a sequence of numbers. The numbers in the range are not modifiable. Generally, range is used for repeating a for loop for a specific number of times. To create a range of numbers, we can simply write:

```
r = range(10)
```

Here, the range object is created with the numbers starting from 0 to 9. We can display these numbers using a for loop as:

```
for i in r: print(i)
```

The above statement will display numbers from 0 to 9. We can use a starting number, an ending number and a step value in the range object as:

```
r = range(30, 40, 2)
```

This will create a range object with a starting number 30 and an ending number 39. The step size is 2. It means the numbers in the range will increase by 2 every time. So, the for loop

```
for i in r: print(i)
```

will display numbers: 30, 32, 34, 36, 38. Let's create a list with a range of numbers from 0 to 9 as:

```
lst = list(range(10))
print(lst) # will display: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Sets

A set is an unordered collection of elements much like a set in Mathematics. The order of elements is not maintained in the sets. It means the elements may not appear in the same order as they are entered into the set. Moreover, a set does not accept duplicate elements. There are two sub types in sets:

- set datatype
- frozenset datatype

set Datatype

To create a set, we should enter the elements separated by commas inside curly braces {}.

```
s = {10, 20, 30, 20, 50}
print(s) # may display {50, 10, 20, 30}
```

Please observe that the set 's' is not maintaining the order of the elements. We entered the elements in the order 10, 20, 30, 20 and 50. But it is showing another order. Also, we repeated the element 20 in the set, but it has stored only one 20. We can use the set() function to create a set as:

```
ch = set("Hello")
print(ch) # may display {'H', 'e', 'l', 'o'}
```

Here, a set 'ch' is created with the characters H,e,l,o. Since a set does not store duplicate elements, it will not store the second 'l'. We can convert a list into a set using the set() function as:

```
lst = [1,2,5,4,3]
s = set(lst)
print(s) # may display {1, 2, 3, 4, 5}
```

Since sets are unordered, we cannot retrieve the elements using indexing or slicing operations. For example, the following statements will give error messages:

```
print(s[0])   # indexing, display 0th element
print(s[0:2])  # slicing, display from 0 to 1st elements
```

The update() method is used to add elements to a set as:

```
s.update([50,60])
print(s) # may display {1, 2, 3, 4, 5, 50, 60}
```

On the other hand, the remove() method is used to remove any particular element from a set as:

```
s.remove(50)
print(s) # may display {1, 2, 3, 4, 5, 60}
```

frozenset Datatype

The frozenset datatype is same as the set datatype. The main difference is that the elements in the set datatype can be modified; whereas, the elements of frozenset cannot be modified. We can create a frozenset by passing a set to frozenset() function as:

```
s = {50,60,70,80,90}
print(s) # may display {80, 90, 50, 60, 70}

fs = frozenset(s)  # create frozenset fs
print(fs) # may display frozenset({80, 90, 50, 60, 70})
```

Another way of creating a frozenset is by passing a string (a group of characters) to the frozenset() function as:

```
fs = frozenset("abcdefg")
print(fs) # may display frozenset({'e', 'g', 'f', 'd', 'a', 'c', 'b'})
```

However, update() and remove() methods will not work on frozensets since they cannot be modified or updated.

Mapping Types

A map represents a group of elements in the form of key value pairs so that when the key is given, we can retrieve the value associated with it. The *dict* datatype is an example for a map. The 'dict' represents a 'dictionary' that contains pairs of elements such that the first element represents the key and the next one becomes its value. The key and its value should be separated by a colon (:) and every pair should be separated by a comma. All the elements should be enclosed inside curly brackets { }.

We can create a dictionary by typing the roll numbers and names of students. Here, roll numbers are keys and names will become values. We write these key value pairs inside curly braces as:

```
d = {10: 'Kamal', 11: 'Pranav', 12:  'Hasini', 13: 'Anup', 14:
'Reethu'}
```

Here, d is the name of the dictionary. 10 is the key and its associated value is 'Kamal'. The next key is 11 and its value is 'Pranav'. Similarly 12 is the key and 'Hasini' is it value. 13 is the key and 'Anup' is the value and 14 is the key and 'Reethu' is the value.

We can create an empty dictionary without any elements as:

```
d = {}
```

Later, we can store the key and values into d as:

```
d[10] = 'Kamal'
d[11] = 'Pranav'
```

In the preceding statements, 10 represents the key and 'Kamal' is its value. Similarly, 11 represents the key and its value is 'Pranav'. Now, if we write

```
print(d)
```

It will display the dictionary as:

```
{10: 'Kamal', 11: 'Pranav'}
```

We can perform various operations on dictionaries. To retrieve value upon giving the key, we can simply mention d[key]. To retrieve only keys from the dictionary, we can use the method keys() and to get only values, we can use the method values().

We can update the value of a key, as: d[key] = newvalue. We can delete a key and corresponding value, using del module. For example del d[11] will delete a key with 11 and its value. Figure 3.10 demonstrates these operations:

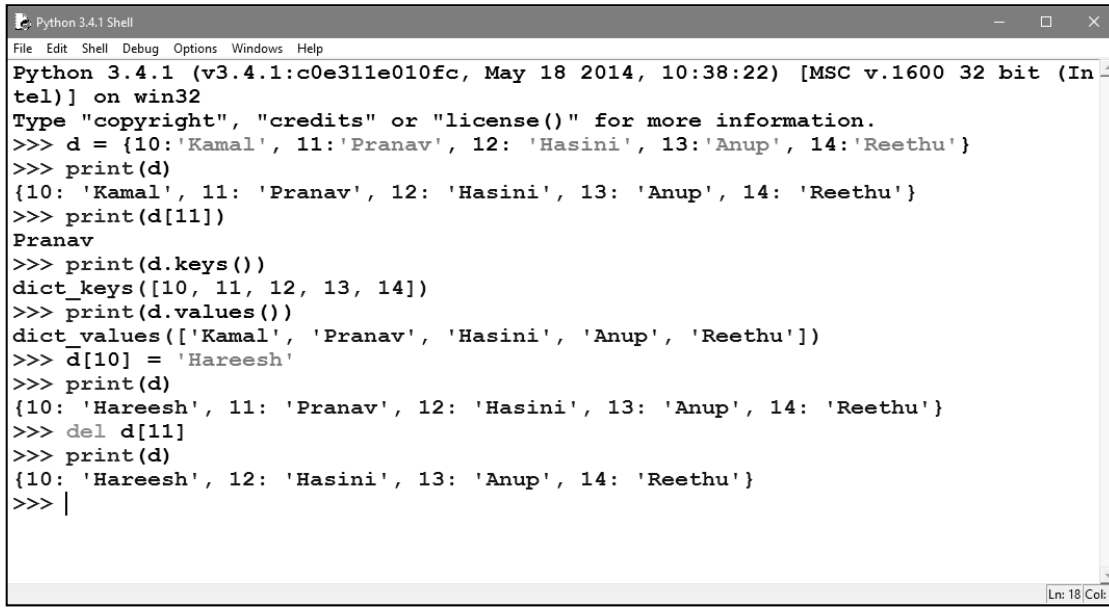


Figure 3.10: Some Operations on Dictionaries

Literals in Python

A *literal* is a constant value that is stored into a variable in a program. Observe the following statement:

```
a  = 15
```

Here, 'a' is the variable into which the constant value '15' is stored. Hence, the value 15 is called 'literal'. Since 15 indicates integer value, it is called 'integer literal'. The following are different types of literals in Python:

- Numeric literals
- Boolean literals
- String literals

Numeric Literals

These literals represent numbers. Please observe the different types of numeric literals available in Python, as shown in Table 3.1:

Table 3.1: Numeric Literals

Examples	Literal name
450, -15	Integer literal
3.14286, -10.6, 1.25E4	Float literal
0x5A1C	Hexadecimal literal
0557	Octal literal
0B110101	Binary literal
18+3J	Complex literal

Boolean Literals

Boolean literals are the True or False values stored into a bool type variable.

String Literals

A group of characters is called a string literal. These string literals are enclosed in single quotes (') or double quotes (") or triple quotes (''' or """). In Python, there is no difference between single quoted strings and double quoted strings. Single or double quoted strings should end in the same line as:

```
s1 = 'This is first Indian book'
s2 = "Core Python"
```

When a string literal extends beyond a single line, we should go for triple quotes as:

```
s1 = '''This is first Indian book on
     Core Python exploring all important
     and useful features'''

s2 = """This is first Indian book on
     Core Python exploring all important
     and useful features"""
```

In the preceding examples, the strings have taken up to 3 lines. Hence, we inserted them inside triple single quotes (''') or triple double quotes (""""). When a string spans more than one line adding backslash (\ ) will join the next string to it. For example,

```
str = "This is first line and \
this will be continued with the first line"
print(str)
This is first line and this will be continued with the first line
```

CHAPTER

17

FILES IN PYTHON

We use different files in our daily life. For example, we may keep all our school certificates in a file and call it 'certificates file'. This file contains several certificates or pages and each page contains some data. From this example, we can simply say that a file is a place for storing data. Similarly, when we go to an office, we can see a book that contains the attendance of the employees. This book is called 'attendance file' since it contains attendance data of employees. Let's understand that if the data is stored in a place, it is called a *file*.

Files

Data is very important. Every organization depends on its data for continuing its business operations. If the data is lost, the organization has to be closed. This is the reason computers are primarily created for handling data, especially for storing and retrieving data. In later days, programs are developed to process the data that is stored in the computer.

To store data in a computer, we need files. For example, we can store employee data like employee number, name and salary in a file in the computer and later use it whenever we want. Similarly, we can store student data like student roll number, name and marks in the computer. In computers' view, a file is nothing but collection of data that is available to a program. Once we store data in a computer file, we can retrieve it and use it depending on our requirements. Figure 17.1 shows the file in our daily life and the file stored in the computer:

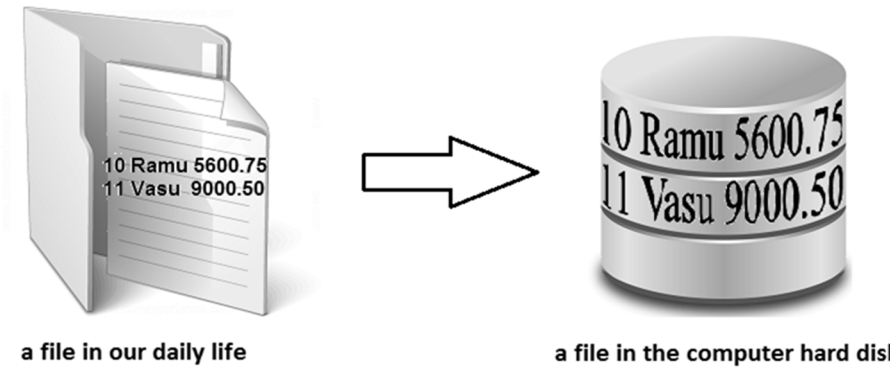


Figure 17.1: A file in our daily life is similar to a file in the computer

There are four important advantages of storing data in a file:

- When the data is stored in a file, it is stored permanently. This means that even though the computer is switched off, the data is not removed from the memory since the file is stored on hard disk or CD. This file data can be utilized later, whenever required.
- It is possible to update the file data. For example, we can add new data to the existing file, delete unnecessary data from the file and modify the available data of the file. This makes the file more useful.
- Once the data is stored in a file, the same data can be shared by various programs. For example, once employee data is stored in a file, it can be used in a program to calculate employees' net salaries or in another program to calculate income tax payable by the employees.
- Files are highly useful to store huge amount of data. For example, voters' list or census data.

Types of Files in Python

In Python, there are two types of files. They are:

- Text files
- Binary files

Text files store the data in the form of characters. For example, if we store employee name "Ganesh", it will be stored as 6 characters and the employee salary 8900.75 is stored as 7 characters. Normally, text files are used to store characters or strings.

Binary files store entire data in the form of bytes, i.e. a group of 8 bits each. For example, a character is stored as a byte and an integer is stored in the form of 8 bytes (on a 64 bit

machine). When the data is retrieved from the binary file, the programmer can retrieve the data as bytes. Binary files can be used to store text, images, audio and video.

Image files are generally available in .jpg, .gif or .png formats. We cannot use text files to store images as the images do not contain characters. On the other hand, images contain pixels which are minute dots with which the picture is composed of. Each pixel can be represented by a bit, i.e. either 1 or 0. Since these bits can be handled by binary files, we can say that they are highly suitable to store images.

It is very important to know how to create files, store data in the files and retrieve the data from the files in Python. To do any operation on files, first of all we should open the files.

Opening a File

We should use open() function to open a file. This function accepts 'filename' and 'open mode' in which to open the file.

```
file handler = open("file name", "open mode", "buffering")
```

Here, the 'file name' represents a name on which the data is stored. We can use any name to reflect the actual data. For example, we can use 'empdata' as file name to represent the employee data. The file 'open mode' represents the purpose of opening the file. Table 17.1 specifies the file open modes and their meanings:

Table 17.1: The File Opening Modes

File open mode	Description
w	To write data into file. If any data is already present in the file, it would be deleted and the present data will be stored.
r	To read data from the file. The file pointer is positioned at the beginning of the file.
a	To append data to the file. Appending means adding at the end of existing data. The file pointer is placed at the end of the file. If the file does not exist, it will create a new file for writing data.
w+	To write and read data of a file. The previous data in the file will be deleted.
r+	To read and write data into a file. The previous data in the file will not be deleted. The file pointer is placed at the beginning of the file.
a+	To append and read data of a file. The file pointer will be at the end of the file if the file exists. If the file does not exist, it creates a new file for reading and writing.

File open mode	Description
x	To open the file in exclusive creation mode. The file creation fails if the file already exists.

Table 17.1 represents file open modes for text files. If we attach 'b' for them, they represent modes for binary files. For example, wb, rb, ab, w+b, r+b, a+b are the modes for binary files.

A buffer represents a temporary block of memory. 'buffering' is an optional integer used to set the size of the buffer for the file. In the binary mode, we can pass 0 as buffering integer to inform not to use any buffering. In text mode, we can use 1 for buffering to retrieve data from the file one line at a time. Apart from these, we can use any positive integer for buffering. Suppose, we use 500, then a buffer of 500 bytes size is used via which the data is read or written. If we do not mention any buffering integer, then the default buffer size used is 4096 or 8192 bytes.

When the open() function is used to open a file, it returns a pointer to the beginning of the file. This is called 'file handler' or 'file object'. As an example, to open a file for storing data into it, we can written the open() function as:

```
f = open("myfile.txt", "w")
```

Here, 'f' represents the file handler or file object. It refers to the file with the name "myfile.txt" that is opened in "w" mode. This means, we can write data into the file but we cannot read data from this file. If this file exists already, then its contents are deleted and the present data is stored into the file.

Closing a File

A file which is opened should be closed using the close() method. Once a file is opened but not closed, then the data of the file may be corrupted or deleted in some cases. Also, if the file is not closed, the memory utilized by the file is not freed, leading to problems like insufficient memory. This happens when we are working with several files simultaneously. Hence it is mandatory to close the file.

```
f.close()
```

Here, the file represented by the file object 'f' is closed. It means 'f' is deleted from the memory. Once the file object is lost, the file data will become inaccessible. If we want to do any work with the file again, we should once again open the file using the open() function.

In Program 1, we are creating a file where we want to store some characters. We know that a group of characters represent a string. After entering a string from the keyboard using input() function, we store the string into the file using write() method as:

```
f.write(str)
```

In this way, write() can be used to store a character or a group of characters (string) into a file represented by the file object 'f'.

Program

Program 1: A Python program to create a text file to store individual characters.

```
# creating a file to store characters
# open the file for writing data
f = open('myfile.txt', 'w')

# enter characters from keyboard
str = input('Enter text: ')

# write the string into file
f.write(str)

# closing the file
f.close()
```

Output:

```
C:\>python create.py
Enter text: This is my first line.
```

In Program 1, characters typed by us are stored into the file 'myfile.txt'. This file can be checked in the current directory where the program is executed. If we run this program again, the already entered data in the file is lost and the file will be created as a fresh file without any data. In this way, new data entered by us will only be stored into the file and the previous data is lost. If we want to retain the previous data and want to add the new data at the end of the file, then we have to open the file in append mode as:

```
f = open('myfile.txt', 'a')
```

The next step is to read the data from 'myfile.txt' and display it on the monitor. To read data from a text file, we can use read() method as:

```
str = f.read()
```

This will read all characters from the file 'f' and returns them into the string 'str'. We can also use the read() method to read only a specified number of bytes from the file as:

```
str = f.read(n)
```

where 'n' represents the number of bytes to be read from the beginning of the file.

Program

Program 2: A Python program to read characters from a text file.

```
# reading characters from file
# open the file for reading data
f = open('myfile.txt', 'r')

# read all characters from file
str = f.read()

# display them on the screen
```

```
print(str)

# closing the file
f.close()
```

Output:

```
C:\>python read.py
This is my first line.
```

If in Program 2, we use the read() method as:

```
str = f.read(4)
```

Then it will read only the first 4 bytes of the file and hence the output will be:

```
This
```

Working with Text Files Containing Strings

To store a group of strings into a text file, we have to use the write() method inside a loop. For example, to store strings into the file as long as the user does not type '@' symbol, we can write while loop as:

```
while str != '@':
    # write the string into file
    if(str != '@'):
        f.write(str+"\n")
```

Please observe the "\n" at the end of the string inside write() method. The write() method writes all the strings sequentially in a single line. To write the strings in different lines, we are supposed to add "\n" character at the end of each string.

Program

Program 3: A Python program to store a group of strings into a text file.

```
# creating a file with strings
# open the file for writing data
f = open('myfile.txt', 'w')

# enter strings from keyboard
print('Enter text (@ at end): ')
while str != '@':
    str = input()   # accept string into str
    # write the string into file
    if(str != '@'):
        f.write(str+"\n")

# closing the file
f.close()
```

Output:

```
C:\>python create1.py
This is my file line one.
This is line two.
@
```

Now, to read the strings from the "myfile.txt", we can use the read() method in the following way:

```
f.read()
```

This method reads all the lines of the text file and displays them line by line as they were stored in the "myfile.txt". Consider the following output:

```
This is my file line one.
This is line two.
```

There is another method by the name readlines() that reads all the lines into a list. This can be used as:

```
f.readlines()
```

This method displays all the strings as elements in a list. The "\n" character is visible at the end of each string, as:

```
['This is my file line one.\n', 'This is line two.\n']
```

If we want to suppress the "\n" characters, then we can use read() method with splitlines() method as:

```
f.read().splitlines()
```

In this case the output will be:

```
['This is my file line one.', 'This is line two.']
```

Program

Program 4: A Python program to read all the strings from the text file and display them.

```
# reading strings from a file
# open the file for reading data
f = open('myfile.txt', 'r')

# read strings from the file
print('The file contents are: ')
str = f.read()
print(str)

# closing the file
f.close()
```

Output:

```
C:\>python read1.py
The file contents are:
This is my file line one.
This is line two.
```

We will plan another program to append data to the existing "myfile.txt" and then to display the data. For this purpose, we should open the file in 'append and read' mode as:

```
f = open('myfile.txt', 'a+')
```

Once the file is opened in 'a+' mode, as usual we can use write() method to append the strings to the file. After writing the data, without closing the file, we can read strings from

the file. But first of all, we should place the file handler to the beginning of the file using seek() method as:

```
f.seek(offset, fromwhere)
```

Here, 'offset' represents how many bytes to move. 'fromwhere' represents from which position to move. For example, 'fromwhere' is 0 represents from beginning of file, 1 represents from the current position in the file and 2 represents from the ending of file.

```
f.seek(10, 0)
```

This will position the file handler at 10th byte from the beginning of the file. So, any reading operation will read data from 10th byte onwards.

```
f.seek(0, 0)
```

This will position the file handler at the 0th byte from the beginning of the file. It means any reading operation will read the data quite from the beginning of the file.

Program

Program 5: A Python program to append data to an existing file and then displaying the entire file.

```
# appending and then reading strings
# open the file for reading data
f = open('myfile.txt', 'a+')

print('Enter text to append(@ at end): ')
while str != '@':
    str = input()   # accept string into str

    # write the string into file
    if(str != '@'):
        f.write(str+"\n")

# put the file pointer to the beginning of file
f.seek(0,0)

# read strings from the file
print('The file contents are: ')
str = f.read()
print(str)

# closing the file
f.close()
```

Output:

```
C:\>python append.py
Enter text to append(@ at end):
This line is added.
@
The file contents are:
This is my file line one.
This is line two.
This line is added.
```

Knowing Whether a File Exists or Not

The operating system (os) module has a sub module by the name 'path' that contains a method isfile(). This method can be used to know whether a file that we are opening really exists or not. For example, os.path.isfile(fname) gives True if the file exists otherwise False. We can use it as:

```
If os.path.isfile(fname):   # if file exists,
    f = open(fname, 'r')    # open it
else:
    print(fname+' does not exist')
    sys.exit()   # terminate the program
```

In Program 6, we are accepting the name of a file from the keyboard, checking whether the file exists or not. If the file exists, then we display the contents of the file; otherwise, we display a message that the file does not exist and then terminate the program.

Program

Program 6: A Python program to know whether a file exists or not.

```
# checking if file exists and then reading data
import os, sys

# open the file for reading data
fname = input('Enter filename: ')

if os.path.isfile(fname):
    f = open(fname, 'r')
else:
    print(fname+' does not exist')
    sys.exit()

# read strings from the file
print('The file contents are: ')
str = f.read()
print(str)

# closing the file
f.close()
```

Output:

```
C:\>python check.py
Enter filename: myfile.txt
The file contents are:
This is my file line one.
This is line two.
This line is added.

C:\>python check.py
Enter filename: yourfile.txt
yourfile.txt does not exist
```

Please observe that while running Program 6, we can enter the same program name 'check.py' to display the source code of the above program.

We can write another program to count the number of lines, words and characters in a text file. The logic is simple. We can use a for loop to read line by line from the file as:

```
for line in f:   # repeat the loop for all lines
    print(line)   # display one line at a time
```

Each line may contain some words. To find out the words, we have to split the line wherever there are spaces. This is done using line.split() method. But this method returns a list with the words.

```
for line in f:
    print(line)
    words = line.split()   # words is a list that contains all words in
                           # a line
    print(words)   # display the list
```

Now, to find out the number of words in a line, we can simply use len() function on 'words' list as len(words). Similarly, to find the number of characters in a line, we can use len(line). So, the total logic will be:

```
for line in f:
    print(line)
    words = line.split()
    print(words)
    cl +=1           # for each line add 1 to cl
    cw += len(words) # in each line add number of words to cw
    cc += len(line)  # in each line, add number of chars to cc
```

Program

Program 7: A Python program to count number of lines, words and characters in a text file.

```
# counting number of lines, words and characters in a file
import os, sys

# open the file for reading data
fname = input('Enter filename: ')

if os.path.isfile(fname):
    f = open(fname, 'r')
else:
    print(fname+' does not exist')
    sys.exit()

# initialize the counters to 0
cl= cw= cc = 0

# read line by line from the file
for line in f:
    words = line.split()
    cl +=1
    cw += len(words)
    cc += len(line)

print('No. of lines: ', cl)
print('No. of words: ', cw)
print('No. of characters: ', cc)
```

```
    # close the file
    f.close()
```

Output:

```
    C:\>python count.py
    Enter filename: myfile.txt
    No. of lines:  3
    No. of words:  13
    No. of characters:  61
```

Working with Binary Files

Binary files handle data in the form of bytes. Hence, they can be used to read or write text, images or audio and video files. To open a binary file for reading purpose, we can use 'rb' mode. Here, 'b' is attached to 'r' to represent that it is a binary file. Similarly to write bytes into a binary file, we can use 'wb' mode. To read bytes from a binary file, we can use the read() method and to write bytes into a binary file, we can use the write() methods.

Let's write a program where we want to open an image file like .jpg, .gif or .png file and read bytes from that file. These bytes are then written into a new binary file. It means we are copying an image file as another file. This is shown in Program 8.

Program

Program 8: A Python program to copy an image file into another file.

```
    # copying an image into a file
    # open the files in binary mode
    f1 = open('cat.jpg', 'rb')
    f2 = open('new.jpg', 'wb')

    # read bytes from f1 and write into f2
    bytes = f1.read()
    f2.write(bytes)

    # close the files
    f1.close()
    f2.close()
```

Output:

```
    C:\>python copy.py
    C:\>
```

When the above program is run, it will copy the image file 'cat.jpg' into another file 'new.jpg'. In this program, our assumption is that the file 'cat.jpg' is already available in the current directory. Current directory is the directory where our program is running. Suppose, if the cat.jpg is not available in the current directory, but it is in another directory, then we have to supply the path of that directory to open() function, as:

```
    f1 = open('c:\\rnr\\cat.jpg', 'rb')
```